

Cryptography Library Benchmark

Bence Nagy

Performance-optimized computing
homework assignment 2025/2026/2

Goal

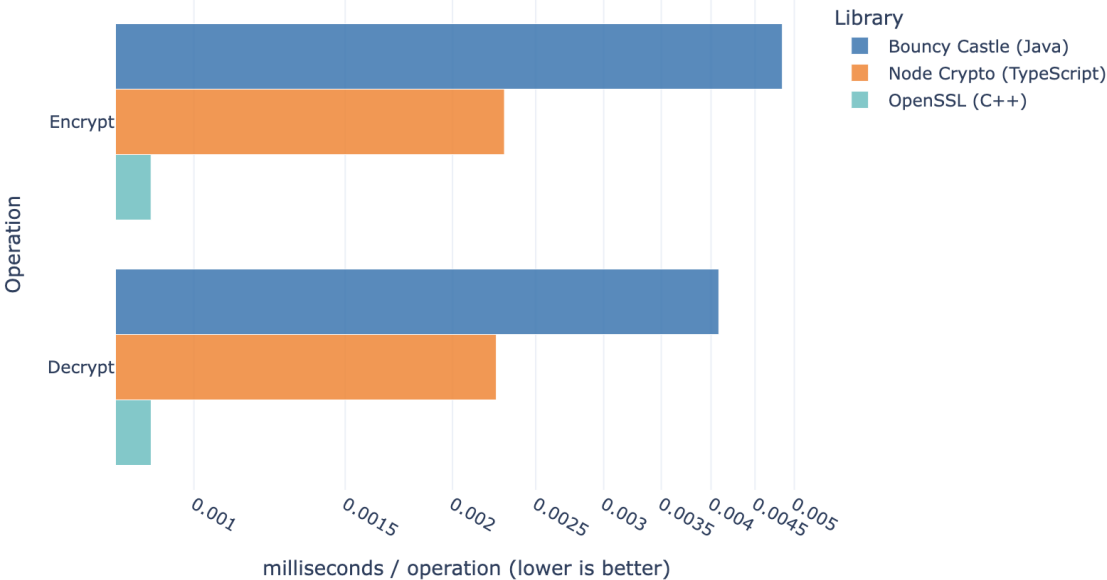
- Measure the performance of cryptographic libraries
- Libraries measured:
 - Bouncy Castle (in Java)
 - Node's Crypto module (in TypeScript)
 - OpenSSL (in C++)
 - (+ A naive implementation of ChaCha20-Poly1305 in C++)
- Cryptographic algorithms:
 - SHA-256 (hashing algorithm)
 - RSA (asymmetric key encryption + decryption)
 - ChaCha20-Poly1305 (symmetric key encryption + decryption)

Benchmarking frameworks used

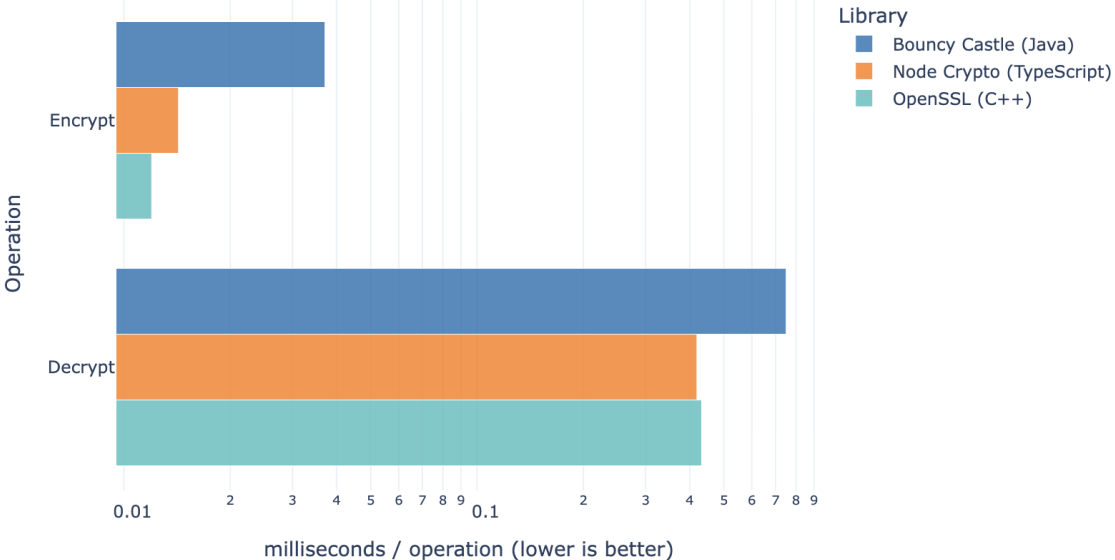
- Java: Java Microbenchmark Harness
- TypeScript: tinybench
- C++: Google benchmark
- The results were exported to JSON files
- The JSON results were aggregated in a Jupyter Notebook file and visualized with plotly
- Final document created with Jupyter Notebook + Quarto

Library results

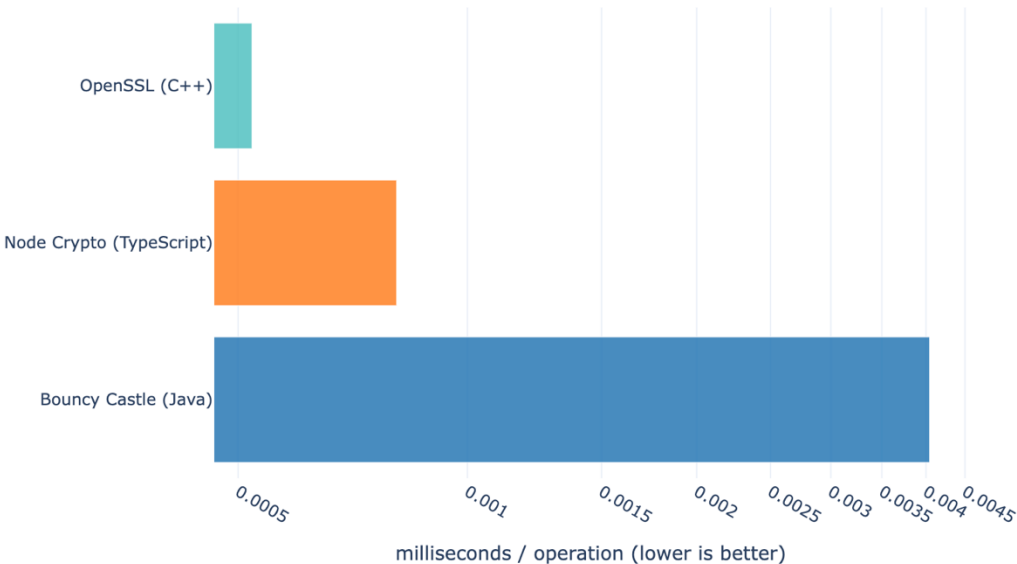
ChaCha20-Poly1305 Encryption and Decryption Runtimes



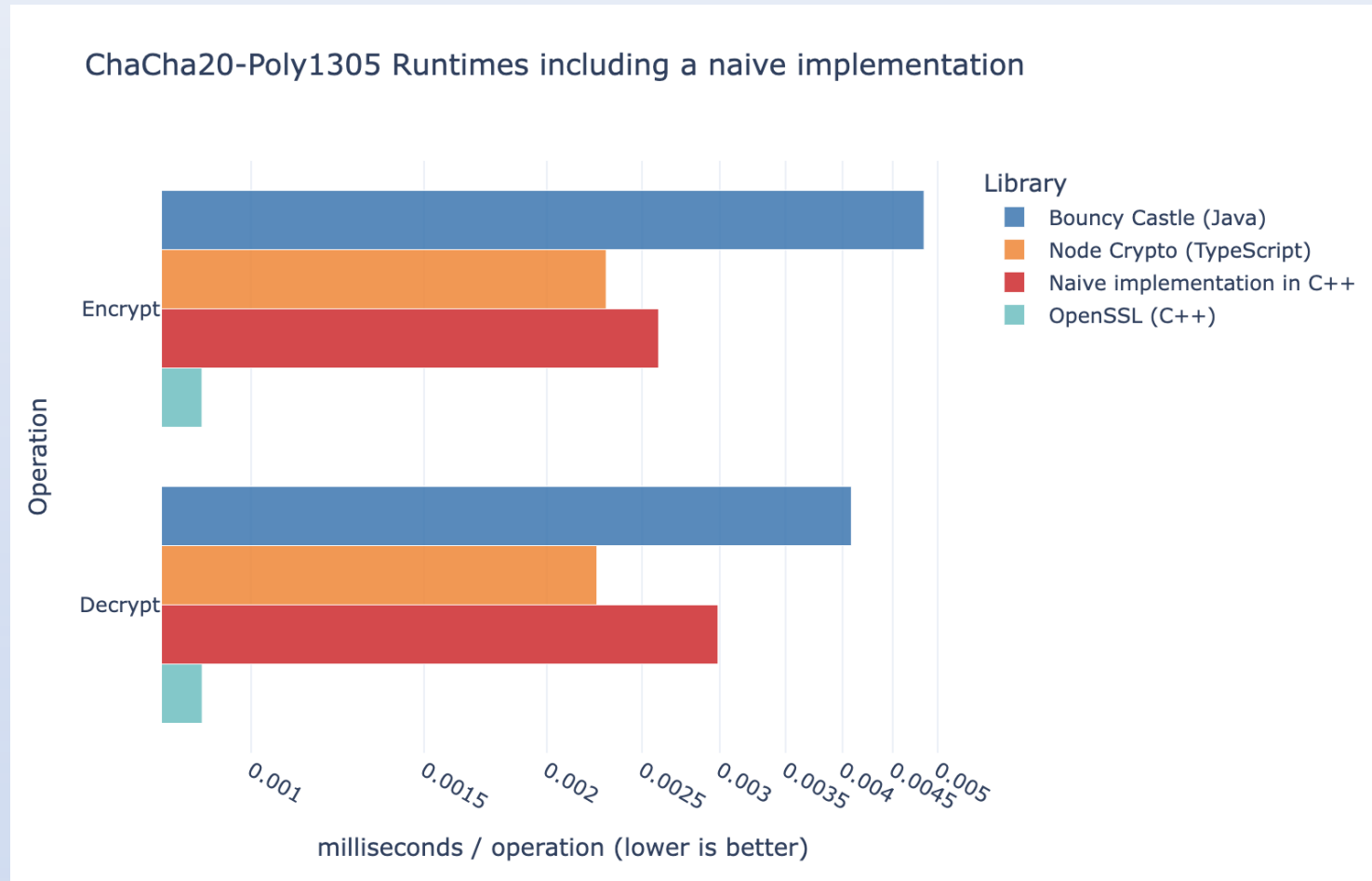
RSA Encryption and Decryption Runtimes



SHA-256 Hashing Runtimes



Naive C++ implementation



Conclusion

- Fastest: OpenSSL (partially written in Assembly using platform specific instruction)
- Second: Node (because it uses OpenSSL under the hood)
- Third: Bouncy Castle (pure Java)
- Naive C++ ChaCha20-Poly1305: slightly slower than Node (3rd fastest overall from 4)